

C++ Programming: Problem Solving and Programming

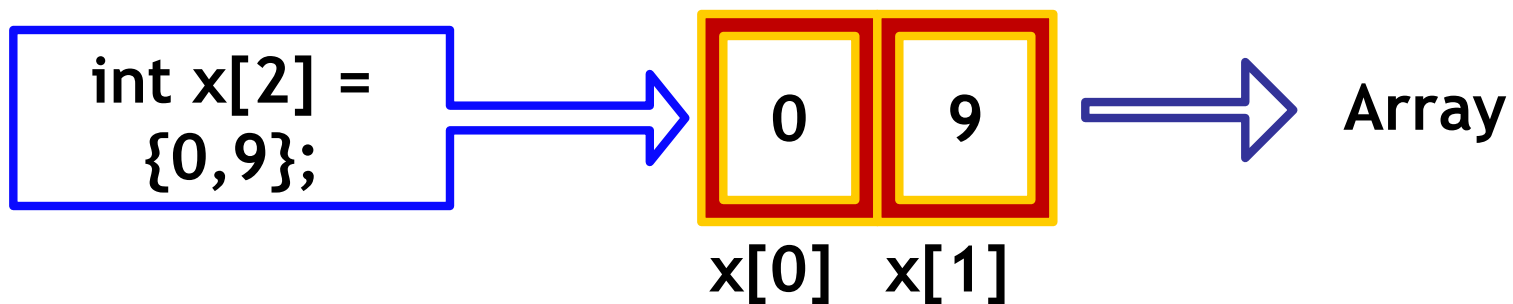
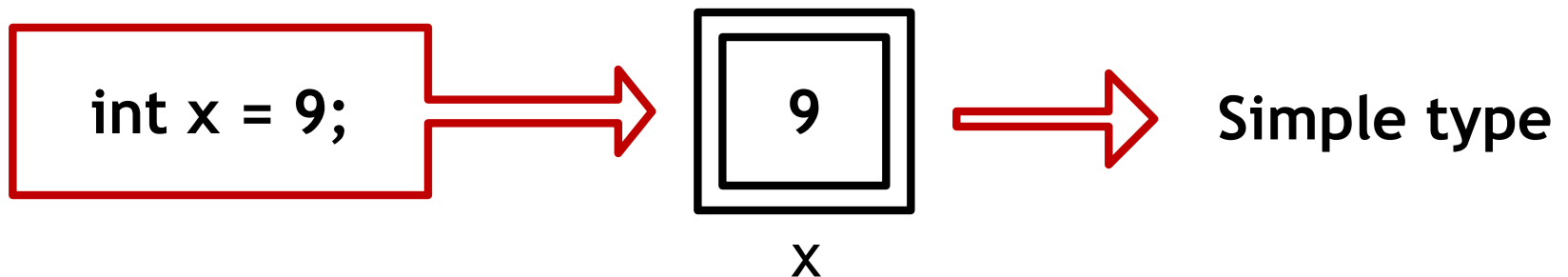
Chapter 8 User-defined Types

Objectives

- **Simple** type
 - ✓ Primitive type (built-in)
 - ✓ User-defined type
 - *typedef* statement
 - *enumeration* types
- **Complex** type
 - ✓ structure (*struct*) / Record
 - ✓ various operations on a struct
 - ✓ relationship between a struct and functions
 - ✓ arrays in a struct

Simple Type

- A data type in which each value is atomic (indivisible)



Primitive / Built-in type

- Almost all programming language provide a set of primitive data types
- They are built into the language

C++ Primitive Type

Type	Primitive Type	Byte
integral	bool	1
	char	1
	short	2
	int	4
	long	4
floating	float	4
	double	8
void	void	-

User-defined Types

- Allows programmers to create new data types, tailored to meet the needs of a particular program.

typedef

- To introduce a new name for an existing type

Enumeration Type (enum)

- To define a new type by listing the literal (values) that make up the type

typedef

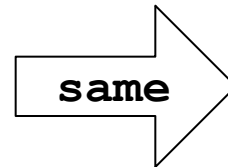
- It merely creates an **additional name** for an existing data type or an *alias*
- It **does not create** a new data type
- Syntax:

typedef ExistingType NewName

typedef - Example

- `typedef int Status;`
`const int GOOD = 1;`

`Status condition;`
`condition = GOOD;`



`int condition;`
`condition = 1;`

typedef - Question

- Modify the code below to use **typedef** Days as integer type and define memory constants for Monday (MON) to Sunday (SUN).

```
int weekday[5] = {1,2,3,4,5};
```

```
int restday[2] = {6,7};
```

```
// Assume 1 represents Monday, 2 represents Tuesday  
and so forth.
```

typedef - Answer

```
typedef int Days;  
const int MON = 1;  
const int TUE = 2;  
const int WED = 3;  
const int THU = 4;  
const int FRI = 5;  
const int SAT = 6;  
const int SUN = 7;  
Days weekday[5] = {MON, TUE, WED, THU, FRI};  
Days restday[2] = {SAT, SUN};
```

typedef - Question

- In an airline booking system, each passenger record contains the passenger's full name, age, and seat number. Define a **typedef** for a struct called **Passenger**, and create a function that prints the list of passengers for a specific flight.

typedef - Answer

```
typedef struct {  
    string name;  
    int age;  
    string  
    seatNumber;  
} Passenger;
```

```
void printPassenger (vector <Passenger>  
    passengers) {  
    cout << "Passenger List:\n";  
    for (Passenger p : passengers)  
        cout << "Name: " << p.name  
            << ", Age: " << p.age  
            << ", Seat: " << p.seatNumber; }
```

```
int main() {  
    vector<Passenger> flight101 = {  
        {"Ah Chin", 34, "12A"}, {"Ah Lam", 45, "12B"},  
        {"Ah Wong", 29, "13A"} };  
    printPassenger(flight101);  
    return 0; }
```

Enumeration type (enum)

- A user-defined data type whose domain is an **ordered** set of **literal values (enumerators)** expressed as **identifiers**.

literal value, a.k.a. enumerator
must be unique in the scope

- Example:

```
enum Days {MON, TUE, WED, THU, FRI, SAT, SUN} ;
```

Days is a
new data
type

Literal values must
be identifier, not
numbers

enumerators are
ordered according
to sequence

enum type - Examples

```
enum Months { JAN , FEB , MAR , APR , MAY , JUN } ;  
enum Days { MON , TUE , WED , THU , FRI , SAT , SUN } ;  
enum Time { MORNING , NOON , EVENING , NIGHT } ;
```

```
enum Faculty { FOCS , FASH , FEBE , FSSH , FAFB } ;  
enum Program { RMM , RSD , RSF , RSD , REI , RIT } ;  
enum Tutorial { G1 , G2 , G3 , G4 , G5 , G6 , G7 } ;
```

- MON, TUE, WED... are identifiers
- All enumerators within the same scope are unique and according to certain sequence

enum type - Question

Are the following *enum* type declarations valid?

1. `enum Vowel {'A', 'E', 'I', 'O', 'U'};`
2. `enum Prizes {1st, 2nd, 3rd};`
3. `enum Animals {MOUSE, RABBIT, COW, TIGER};`
`enum Pets {CAT, DOG, BIRD, FISH, RABBIT};`

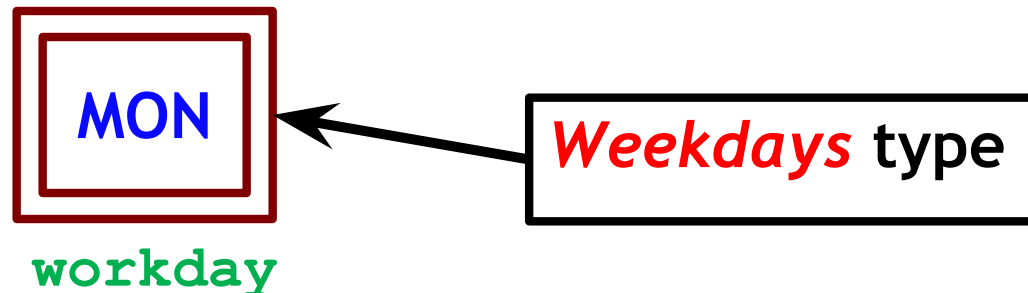
enum type - Variables

- To define and initialize the variables of *enum* type

EnumType VariableName;

- Example:

```
enum Weekdays {MON, TUE, WED, THU, FRI} ;  
Weekdays weekday = MON;
```



enum type - Operation

- Declaration of the enum type and its variable

```
enum Weekdays {MON,TUE,WED,THU,FRI};  
Weekdays workday;
```

- To assign MON as the value of workday variable

```
workday = MON;
```

- To read the value of the workday variable

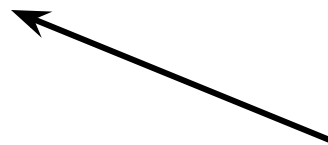
```
If ( workday == MON )  
    cout << "Monday Blue";
```

enum type - Operation

- Display the value of *enum* type variable will obtain the **index** of enumerator
- Example:

```
enum Weekdays {MON, TUE, WED, THU, FRI};  
Weekdays workday = MON;  
cout << workday;
```

The output is 0



index of MON = 0

enum type - Operation

- **Invalid** enum type operation

```
enum Weekdays {MON, TUE, WED, THU, FRI} ;  
Weekdays workday = MON;
```

```
workday = 1; //attempt to assign index 1 to workday
```

Implicit type coercion from an integral type to an enumeration type is invalid.

```
workday++; //attempt to increase the value of workday
```

workday = workday + 1; // similar error as above

enum type - Operation

- **Correct** enum type operation

```
enum Weekdays {MON, TUE, WED, THU, FRI};  
Weekdays workday = MON;
```

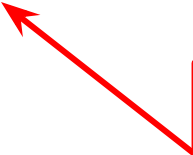
```
workday = Weekdays(1);
```

this is valid



```
workday = Weekdays(workday + 1);
```

this is valid. It increases the current
workday by 1



enum type - Question

- Evaluate the output

```
enum Weekdays {MON, TUE, WED, THU, FRI};
enum Emotion {SAD, HAPPY, ANGRY, EXCITED};
Weekdays workday = FRI;
Emotion mood = SAD;

if(workday < WED)
    mood = SAD;
else if(workday == WED)
    mood = HAPPY;
else
    mood = EXCITED;
cout << "Mood = " << mood;
```

What is the output? **Mood = 3**

enum type - Question

- Assume that a restaurant sells a few types of fast food. They defined the food categories using the codes below:

Type	Code
Fried Food	FF
Burger	BG
Pizza	PZ

Fast Food

- The items for each category are coded as below:

Type	Code
Fried Chicken	FFC
Fried Nugget	FFN
French Fries	FFF

Fried Food

Type	Code
Chicken Burger	BGC
Beef Burger	BGB
Fish Burger	BGF

Burger

Type	Code
Chicken Pizza	PZC
Vege Pizza	PZV
Tuna Pizza	PZT

Pizza

enum type - Question

- Define `Fastfood`, `Friedfood`, `Burger` and `Pizza` as shown above as enum types.
- Create 4 enum type variables to store the `Fastfood` type, `Friedfood` type, `Burger` type and `Pizza` type values. Initialize them with the respective first item values and display them.
- The code below is to obtain an index from the user. E.g. If the user enters 1, BG should be assigned to the correspondent `Fastfood` type variable (defined above). Complete the code.

```
int index ;  
cin >> index; // assume that the user entered 1  
// continue your code here
```

enum type - switch

- Since the enumerators are literals, they can appear in case labels. For example:

```
enum Emotion {SAD, HAPPY, ANGRY, EXCITED};  
Emotion mood = ANGRY;  
switch(mood) {  
    case SAD:  
        cout << "Don't worry, be happy";  
        break;  
    case HAPPY: case EXCITED:  
        cout << "Keep being happy!";  
        break;  
    case ANGRY:  
        cout << "Forgive, and you'll forget";  
}
```

enum type (switch) - Question

- Continue from the exercise in Slide 22, use switch case to assign the respective price as shown below to a variable named `price`.

Type	Code	Price (MYR)
Chicken Burger	BGC	5.80
Beef Burger	BGB	5.50
Fish Burger	BGF	6.80

Burger

enum type - function

- Return an *enum* type value from a

```
enum Weekday {MON,TUE,WED,THU,FRI};
Weekday stringToDay(string);
int main()
{   string day = "Monday";
    Weekday workday = stringToDay(day);
    return 0;
}
Weekday stringToDay(string d)
{   if( d == "Monday")
        return MON;
    else if (d == "Tuesday")
        return TUE;
    ...
}
```

enum type - Question

- Fill in the blank:

```
enum Mood {SAD,HAPPY};  
__①__  
int main()  
{  
    int x = 1;  
    __②__ ans= getMood(1);  
    return 0;  
}
```

```
__③__ getMood(__④__ m)  
{  
    Mood emotion;  
    if(m==0)  
        emotion = SAD;  
    else  
        emotion = HAPPY;  
    return emotion;  
}
```